1. Instruction Set Architecture & Compiler Basics

2026 Fall

Hunjun Lee
hunjunlee@hanyang.ac.kr

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

OK, very basics first.

“How does your program run on computer?”

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Basic components of a computer

- To get a task done by a general-purpose computer, we need
  - A computer program
    - Specifies what computers must do!
  - The computer itself
    - To carry out the specified task

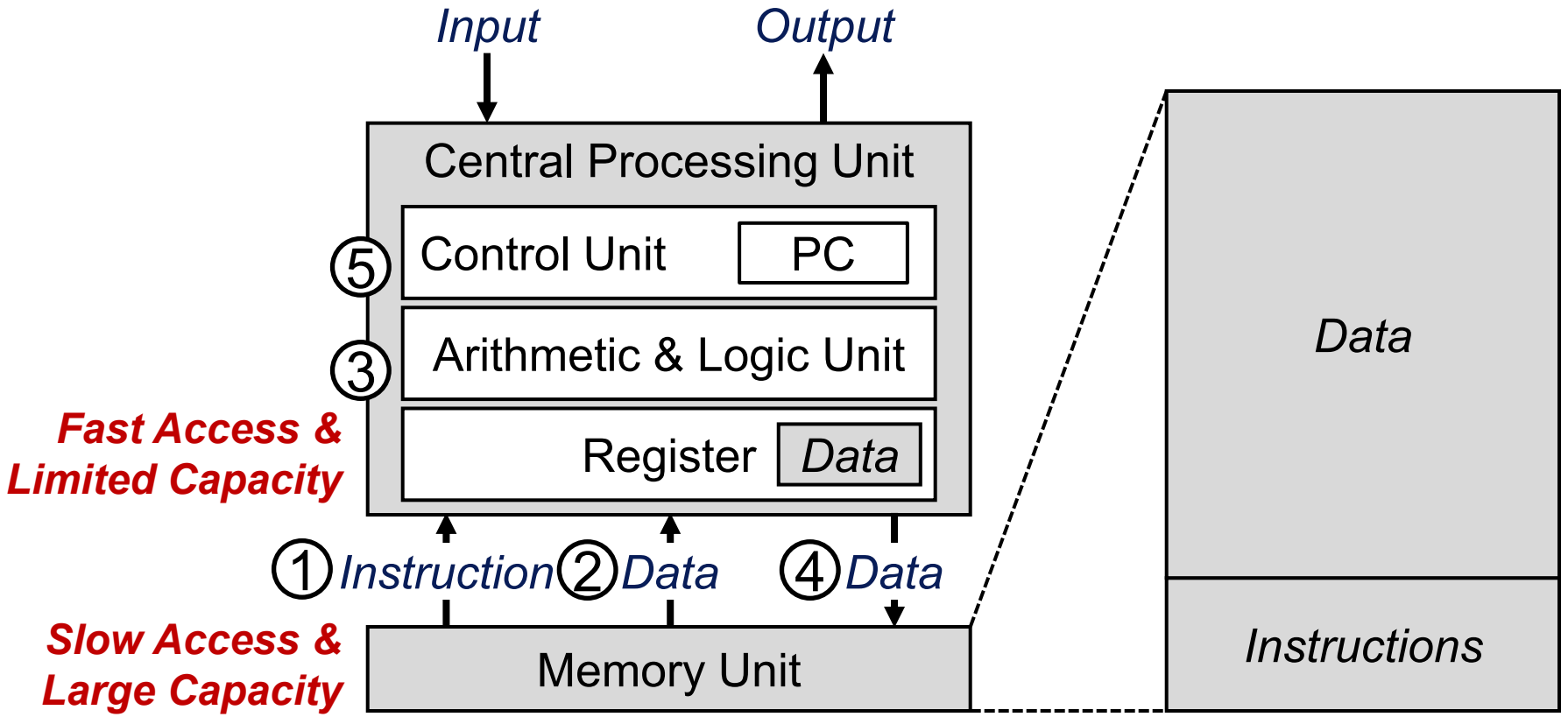
- Program: A sequence of instructions
  - Instruction: the smallest piece of specified work that the computer can carry out

- Instruction set: All possible instructions that a computer is designed to be able to carry out

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Von Neumann Architecture

- Both instructions and data are stored in the memory
- Instructions dictate (1) which and how data are manipulated and (2) which instruction should be next

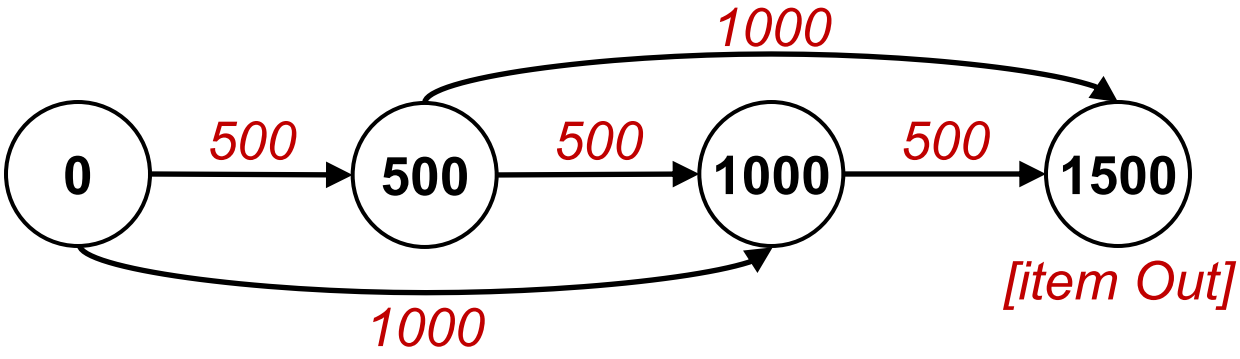


© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Recall: State machine

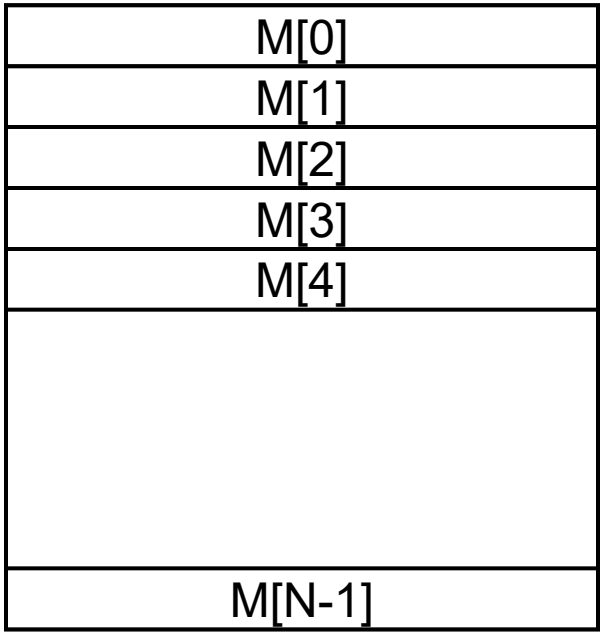
- Computers are essentially a complex state machine

- State machine (Ex. vending machine)
  - Specification: When a user inputs a total of 1500 ₩, the machine outputs a Coke
  - Input type: a user can input 500 ₩ coin and 1000 ₩ bill)
  - States: an amount of cash a user has inputted



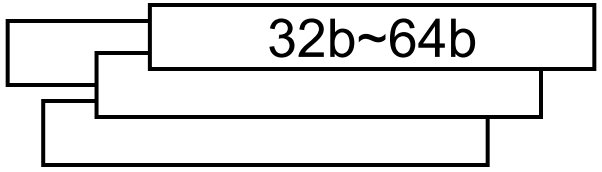
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Programmer visible state
(a.k.a. architectural state)



Memory

Array of storage locations
indexed by an address



Registers

- Given special names in the ISA
  (as opposed to addresses)
- General vs. special purpose

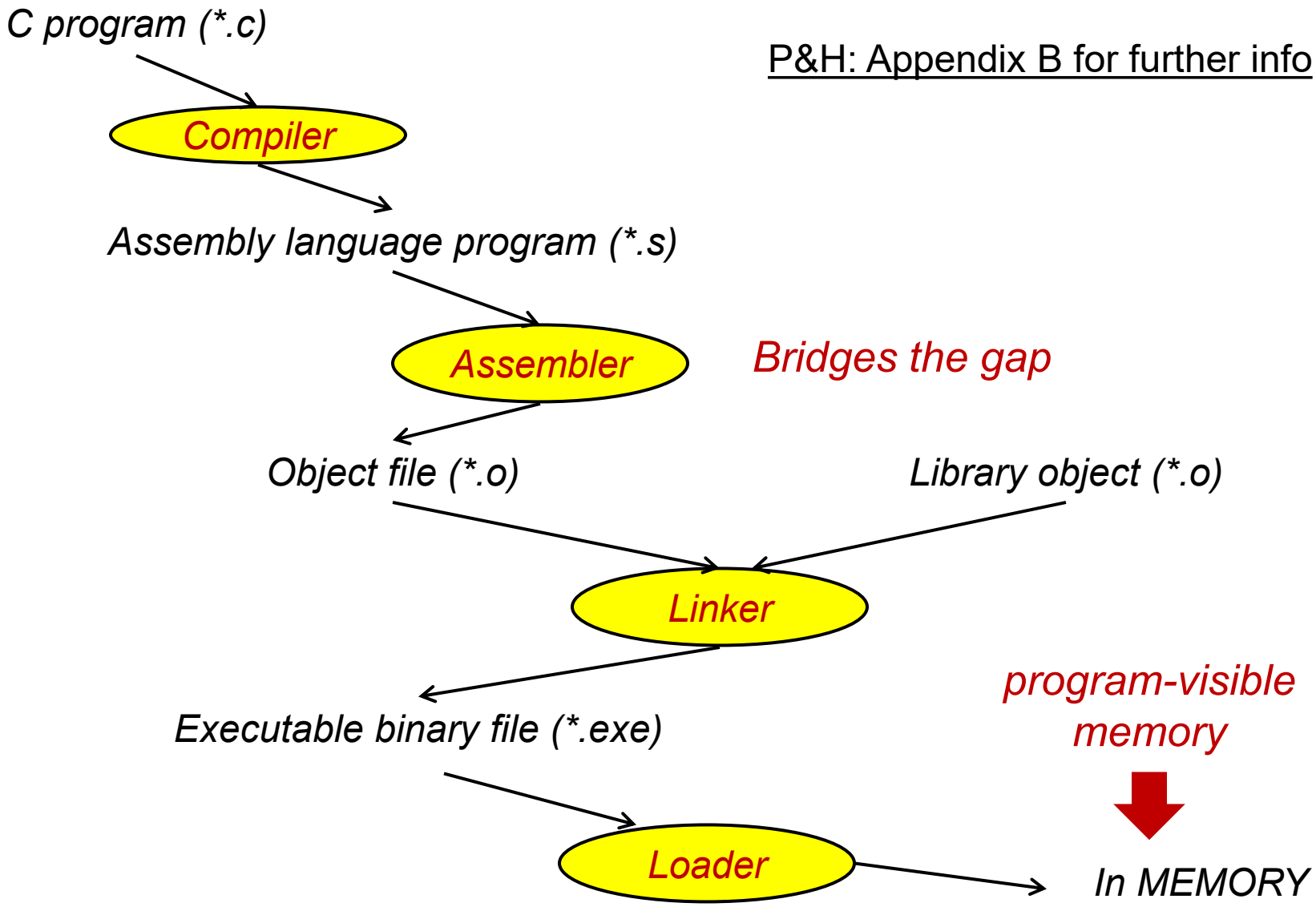
Program Counter (32b~64b)

Memory address
of the current instruction

Instructions (and programs) specify how to transform
the values of programmer visible state

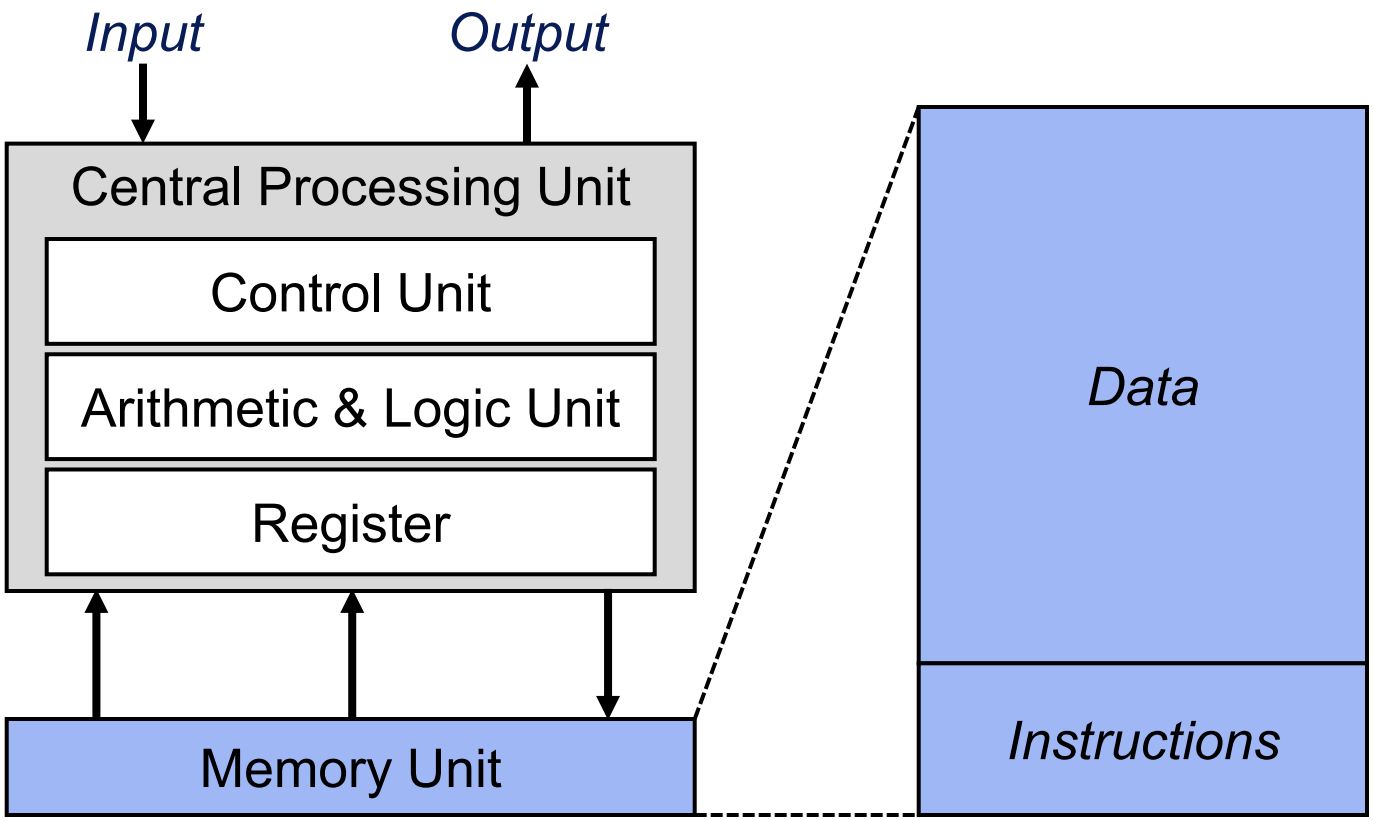
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

How to load a program to your computer?



© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Von Neumann Architecture



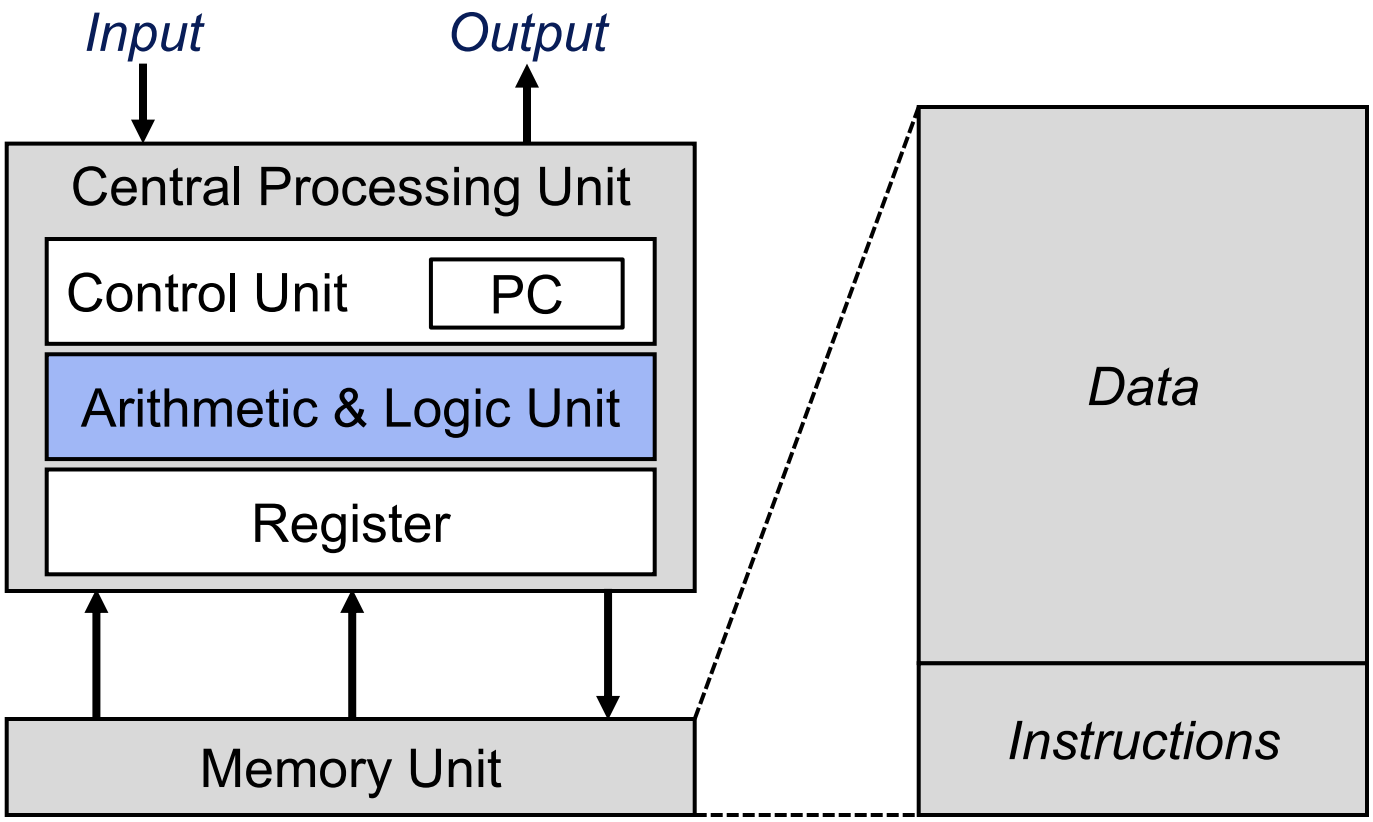
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Runtime storage organization

- The memory stores the following:
  - (1) program (instructions) and (2) data required in program execution
- Memory contains bits
  - Logically grouped into bytes (8 bits) and words (e.g., 8, 16, 32 bits)
  - The word size determines the instruction width, register size, …
- Address space: Total number of uniquely identifiable locations in memory (differs depending on the architecture)
  - In LC-3, the address space is 2^{16} (16-bit addresses)
  - In MIPS, the address space is 2^{32} (32-bit addresses)
  - In x86-64, the address space is (up to) 2^{48} (48-bit addresses)

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Von Neumann Architecture



© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Processing units

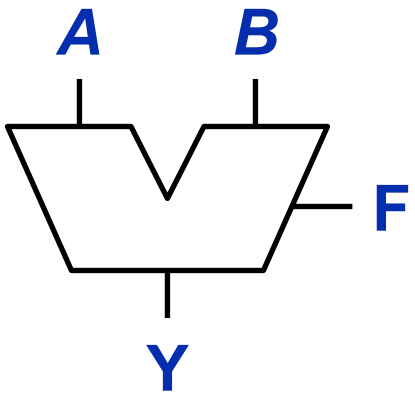
- Performs the actual computation(s)
- The processing unit can consist of many functional units
- We start with a simple Arithmetic and Logic Unit (ALU), which executes computation and logic operations
  - MIPS: add, sub, mult, and, nor, …
- The ALU processes quantities that are referred to as words
  - Word length in MIPS is 32 bits

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

ALU (Arithmetic logical unit)

- Combines a variety of arithmetic and logical operations into a single unit (that performs only one function at a time)

- Usually denoted with this symbol:



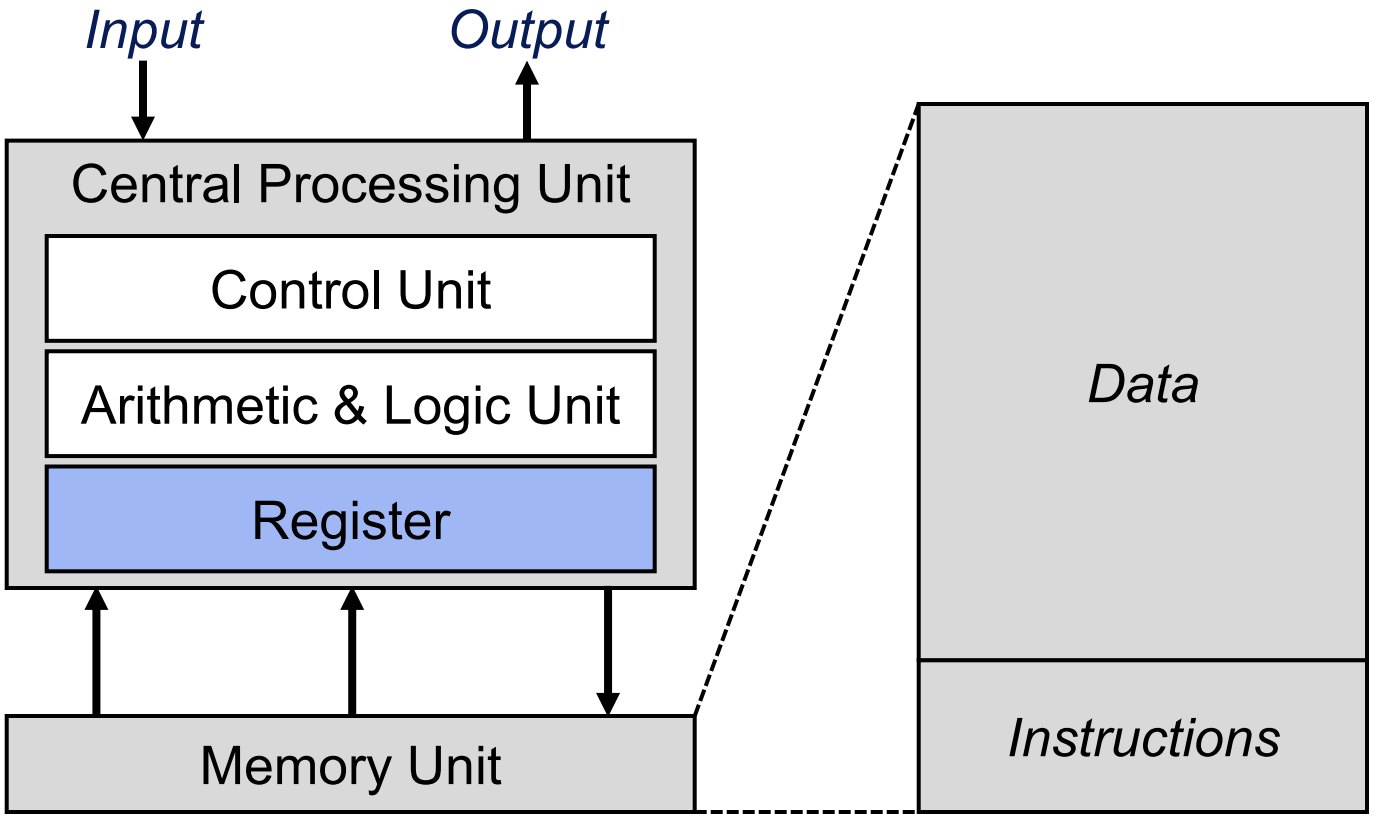
F[2:0]	Function
000	Y = A and B
001	Y = A or B
010	A + B
011	Not used
100	A – B
101	A * B
110	A / B
111	SLT

One of the examples

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Von Neumann Architecture

- Both instructions and data are stored in the memory
- Each instruction dictates (1) which and how data are manipulated and (2) which instruction should be next



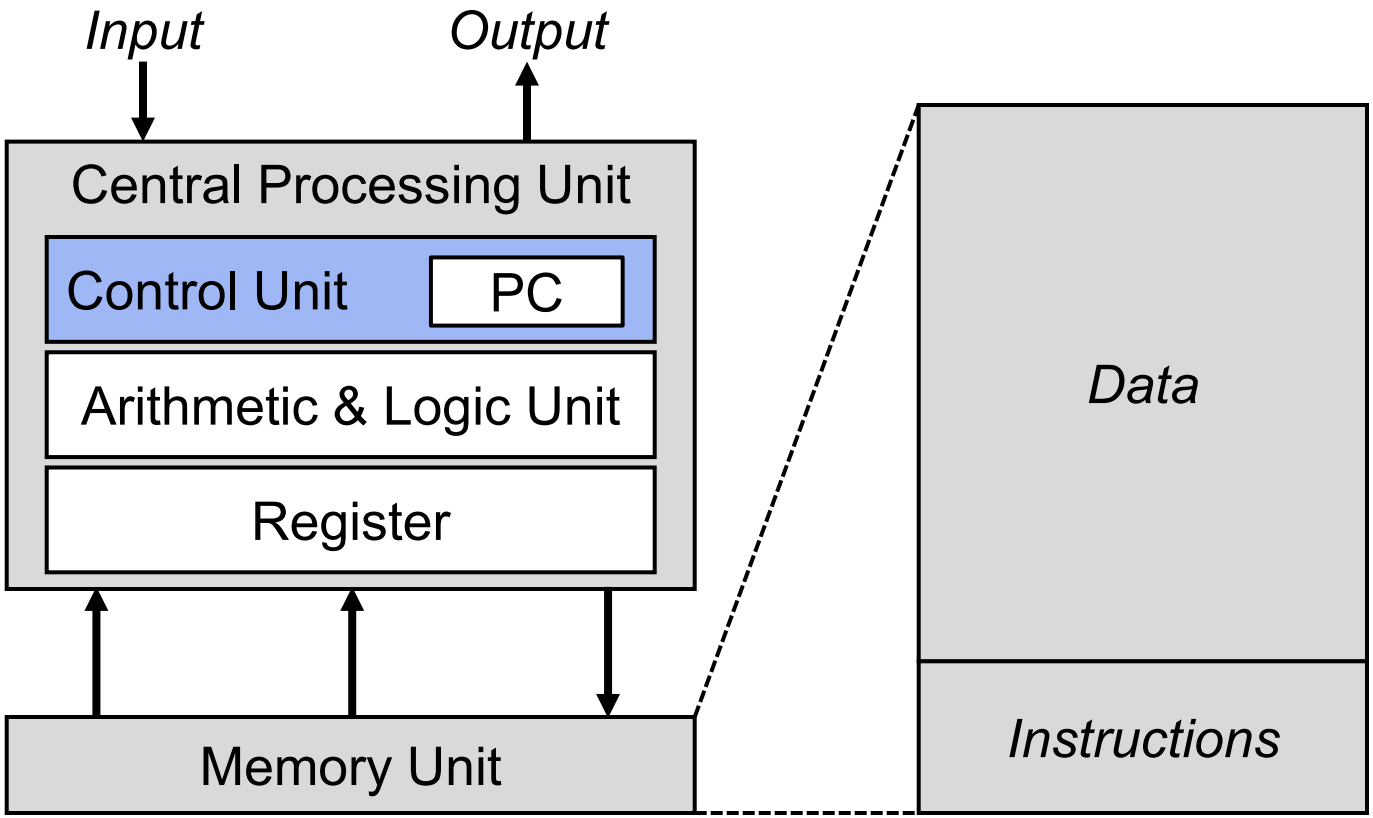
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Registers: fast storage

- **Memory** is large but slow while **registers** are fast and small
- Processing unit utilizes registers during operation
  - Ensure fast access to values to be processed in the ALU (stores temporary values)
  - Combinational read & synchronous write ➔ can execute read + ALU + write in a single cycle
- Register Set or Register File
  - Set of registers that can be manipulated by instructions
  - MIPS has 32 registers
    - R0 to R31: 5-bit register number (or Register ID)
    - Register size = Word length = 32 bits
  - There are some special-purpose registers (e.g., `$fp, $sp, …`)

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Von Neumann Architecture



© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Control unit

- Enables a step-by-step execution of a program
  - Proceeds through each instruction in a program in sequence
- Keeps track of which instruction is being processed, via an instruction register
- Keeps track of which instruction to process next, using a program counter (PC) and determines which instruction to process next

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

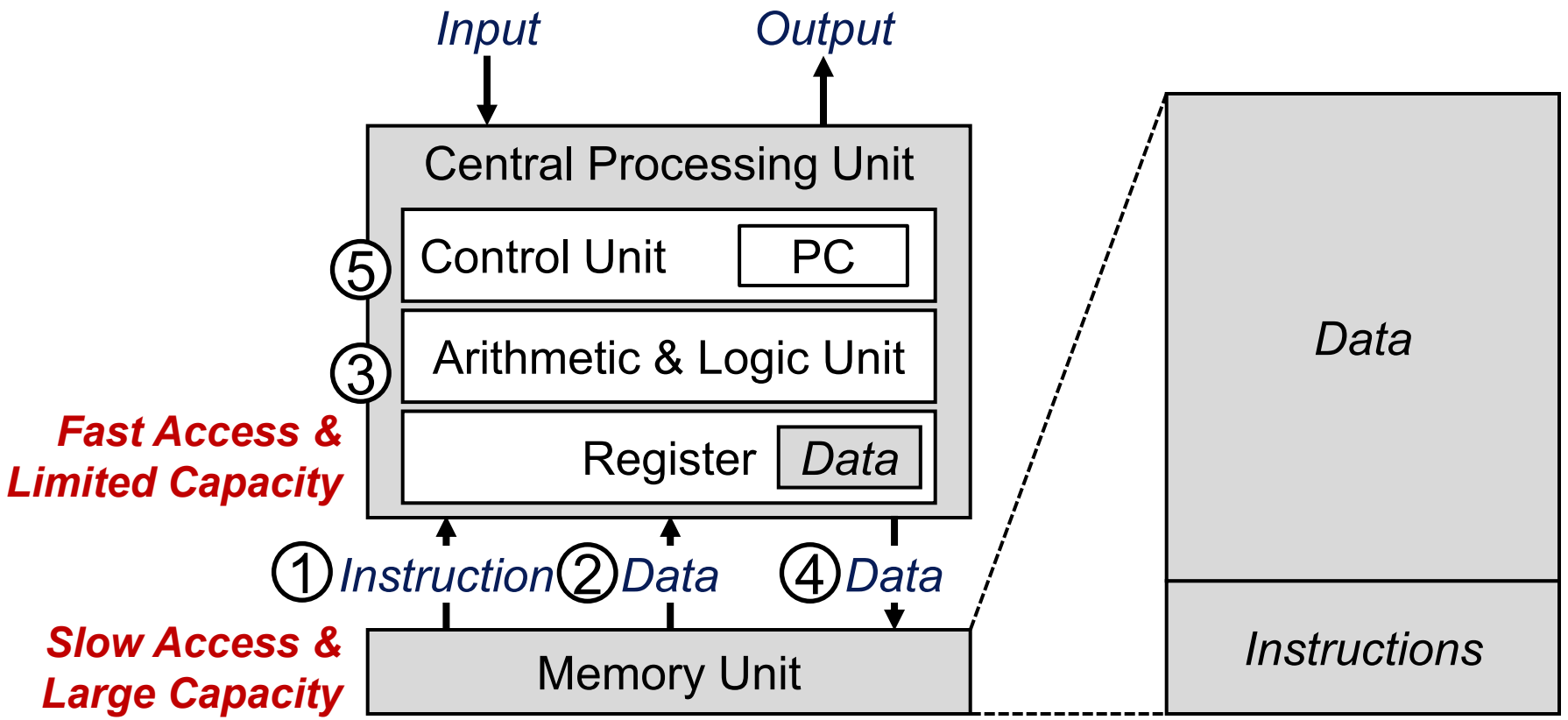
Wrap up

- **Memory Unit:** stores (1) program (instructions) and (2) data required in program execution
- **Arithmetic & Logical Unit:** performs the actual computation(s)
- **Control Unit:** enables a step-by-step execution of a program

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Von Neumann Architecture

- Instructions dictate how these components should be used to change the programmable visible states!



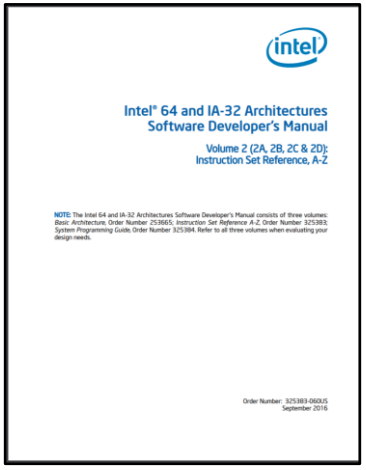
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

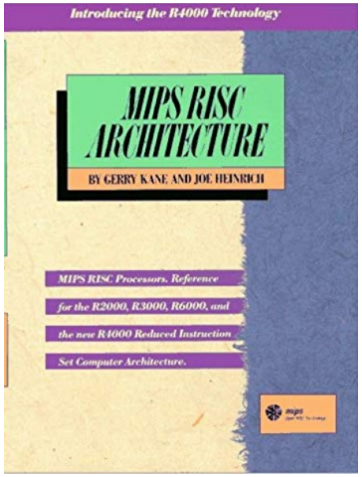
Instruction Set Architecture (ISA)

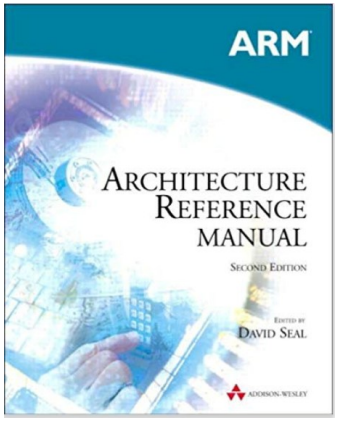
“User’s manual for the computer”

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Architecture Manuals









Each vendor specifies the instruction sets in a different way

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Architecture*

- "The term *architecture* is used here to describe the attributes of a system **as seen by the programmer**, i.e., the conceptual structure and functional behavior, as distinct from the organization of the data flow and controls, the logical design, and the physical implementation."

--- footnote on page 1, Architecture of the IBM System/360,
Amdahl, Blaauw and Brooks, 1964.

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

How to specify what a computer does?

- **Architecture** Level
  - Car: driving manual & operation manual

  // you don't have to be a car mechanic to drive a car.
  - Computer : program manual

  // you don't have to be a circuit designer to program a computer.

- **Microarchitecture** (implementation) Level
  - A particular car design has a certain configuration of electrical/mechanical components (e.g., v8 engine vs. v4 engine)
  - A particular computer design has a certain configuration of datapath and control logic units (e.g., adder type, cache, …)

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

What are specified/decided in an ISA?

- How to change the programmable visible states!
  - Data format and size
    - character, binary, decimal, floating point, …
  - "Programmer Visible State" **(a.k.a. architectural state)**
    - memory, registers, program counter (PC), etc.
  - Instructions: how to **"change"** the programmer visible state?
    - What to perform and what to perform next
    - Where the operands are
  - How to interface with the outside world?
  - Protection and privileged operations
  - Software conventions

Often, you compromise performance for future scalability and compatibility

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

General Instruction Classes

- **Arithmetic and logical operations** (e.g., add, sub, and, or)
  1) Load operands from specified locations
  2) Compute a result as a function of the operands
  3) Store result to a specified location
  4) Update PC to the next sequential instruction
- **Data movement operations** (e.g., load, store)
  1) Fetch operands from specified locations
  2) Store operand values to specified locations
  3) Update PC to the next sequential instruction
- **Control flow operations** (e.g., branch, jump)
  1) Fetch operands from specified locations
  2) Compute a branch condition and a target address
  3) If "branch condition is true" then PC ← target address
     else PC ← next seq. instruction

Generally defined to be atomic

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

RISC ISA

- Simple operations
  - 2-input, 1-output arithmetic and logical operations
  - Only few alternatives exist to do the same thing
- Simple data movements
  - ALU ops are register-to-register (need a large register file)
  - Memory can be accessed by only load and store instructions
    → **"Load-store architecture"**
- Simple branches
  - Limited varieties of branch conditions and targets
- Simple instruction encoding
  - All instructions encoded **in the same number of bits**
  - Only a few formats

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

RISC vs. CISC

- RISC (Reduced instruction set architecture)
  - The hardware exposes only basic operations as an ISA
  - Simpler instruction (+ simple decoding)
  - Fixed-width instruction (one word)

- CISC (Complex instruction set architecture)
  - Exposes more complex operations (combination of multiple RISC-style operations)
  - E.g., A single instruction may … load the data from both memory and register, perform addition, and write back the result
    - Remember `ADD r2, r1, [2000]`

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Evolution of ISA

- Why were the earlier ISAs so simple?
  - Technology limitation
  - Inexperience, lack of precedence
- Why did it get so complicated later?
  - Complex instruction set architecture (CISC)
  - Ease of assembly programming
  - Lack of memory size and performance (in 1970s~80s)
  - Micro-programmed implementation
- Why did it become simple again?
  - Reduced instruction set architecture (RISC)
  - Memory size and speed (cache!)
  - **Compilers**

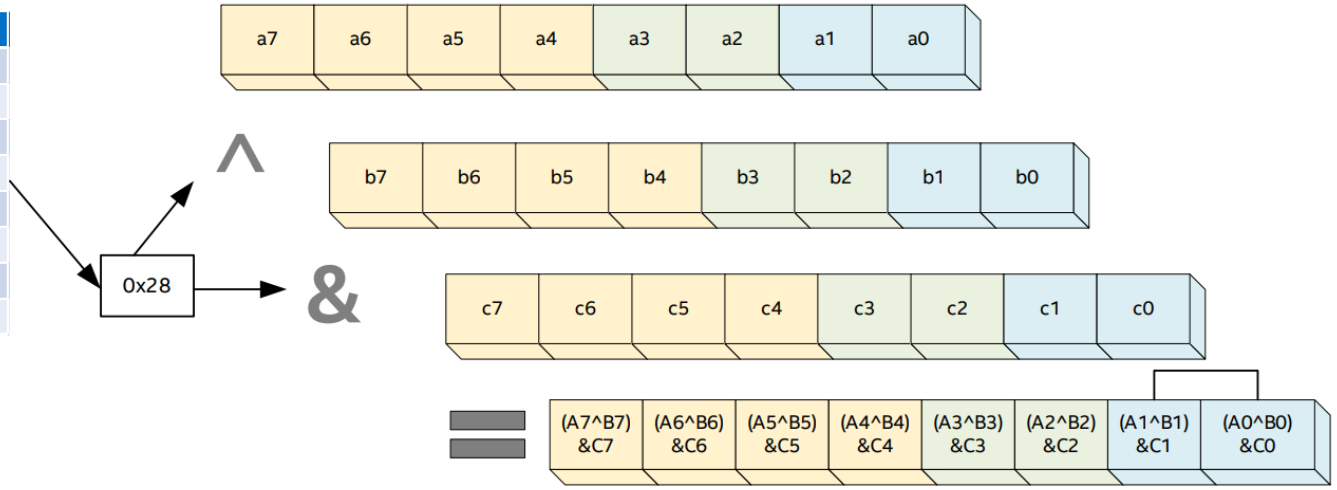
What about x86 (Intel and AMD)?

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

ISA extension in modern processors - 1

- Intel AVX enables vector operations by defining an Intel AVX-512 ISA

Bit	A	B	C	(A ^ B) & C
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	1	0



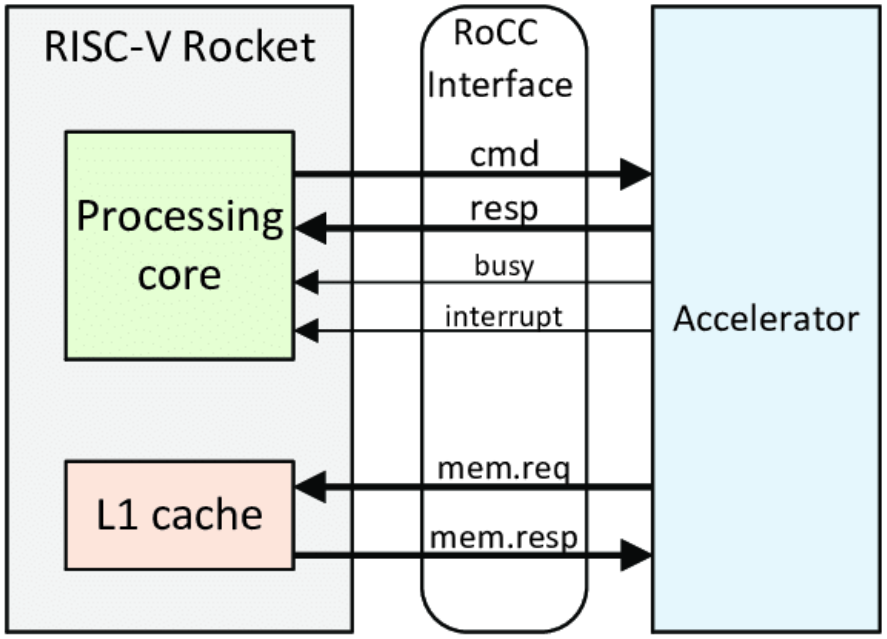
INSTRUCTION SET	C INTRINSIC FORM OF INSTRUCTION
AVX-512	__m512i _mm512_ternarylogic_epi64 (__m512i a, __m512i b, __m512i c, int imm8)
AVX-512	__m512i _mm512_mask_ternarylogic_epi64 (__m512i src, __mmask8 k, __m512i a, __m512i b, int imm8)
AVX-512	__m512i _mm512_maskz_ternarylogic_epi64 (__mmask8 k, __m512i a, __m512i b, __m512i c, int imm8)

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

ISA extension in modern processors - 2

- Rocket custom coprocessor (RoCC) interface in RISC-V
  - The computer architects can add a custom coprocessor
  - The RoCC interface controls the coprocessor

Controls the accelerator using RoCC



Add a custom accelerator (e.g., NPU)

```
1 #define ROCC_INSTRUCTION_0_R_R(x, rs1, rs2, func7){
2       asm volatile(
3       ".insn r " STR(CAT(CUSTOM_, x)) ", "
4       STR(0x3) ", " STR(func7) ", x0, %0, %1”
5       :
6       : "r"(rs1), "r"(rs2));
7   }
```

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Wrap-up: Terminologies

- Instruction Set Architecture (ISA)
  - The machine behavior as observable and controllable by the programmer
- Instruction Set
  - The set of commands understood by the computer
- Assembly Code
  - A collection of instructions expressed in “textual” format
    e.g. Add r1, r2, r3
  - Converted to machine code by an assembler
  - One-to-one correspondence with machine code
- Machine Code
  - A collection of instructions encoded in binary format 0101000…
  - Directly consumable by the hardware

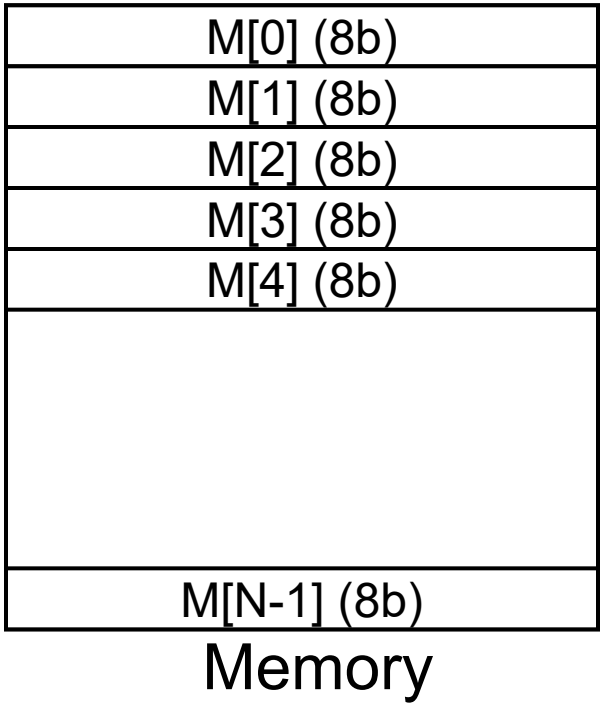
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Let’s dive into a concrete example

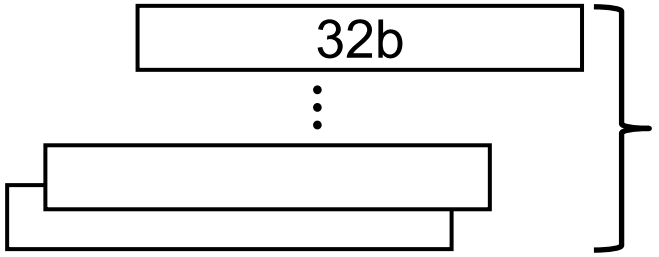
MIPS ISA (32bit)

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

MIPS architectural state



Word addressable



32 entry register file

Register File

Program Counter (32b)

32bit address space + Byte addressable

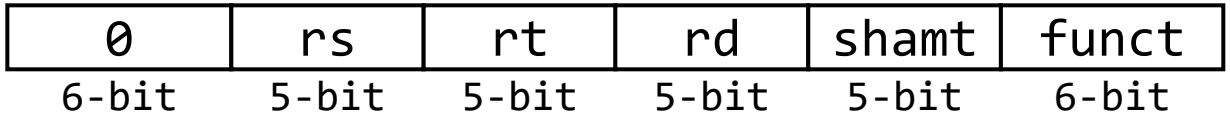
32bit instruction

MIPS architecture has (1) 32-bit word size and (2) a 32-entry register file

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

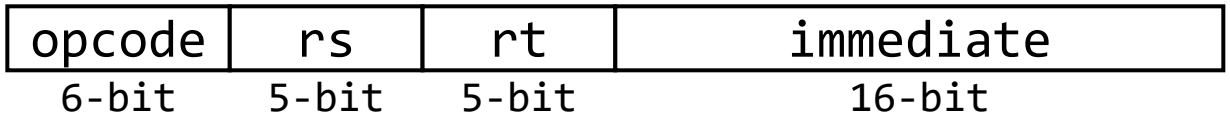
MIPS instruction formats

- Three simple formats
  - R-type, 3 register operands



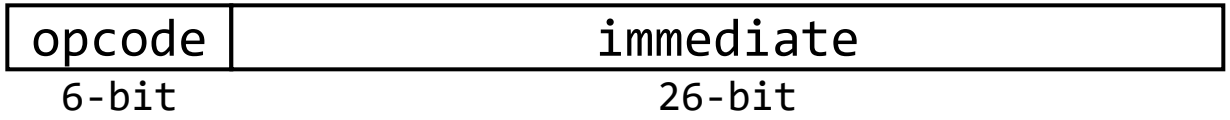
R-type

  - I-type, 2 register operands and 16-bit immediate operand



I-type

  - J-type, 26-bit immediate operand

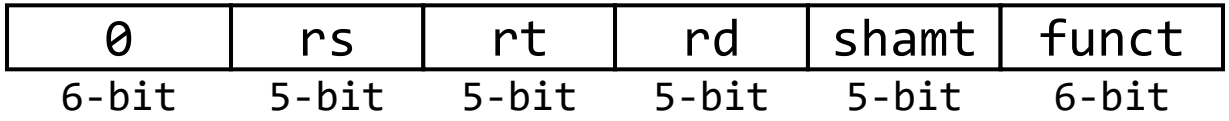


J-type

- Simple Decoding
  - 4 bytes per instruction, regardless of format (fixed size)
  - Must be 4-byte aligned (2 LSB of PC must be 2b'00)
  - Format and fields readily extractable

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

R-type Instructions



R-type

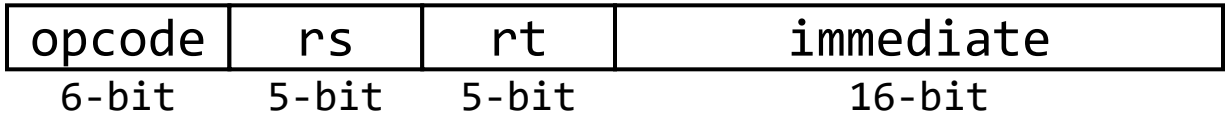
- `funct`: type of ALU operation
  - Arithmetic: {signed, unsigned} x {ADD, SUB, MULT, DIV, …}
  - Logical: {AND, OR, XOR, NOR, …}
  - Shift: {Left, Right-Logical, Right-Arithmetic}

- `shamt`: shift amount (only used for shift operation)

- Assembly and semantics
  - Corresponding assembly: **`opcode rd rs rt`**
    - `RF[rd] = RF[rs] op RF[rt]`
    - `PC = PC + 4 (use the next instruction)`

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

I-type Instructions – ALU ver.

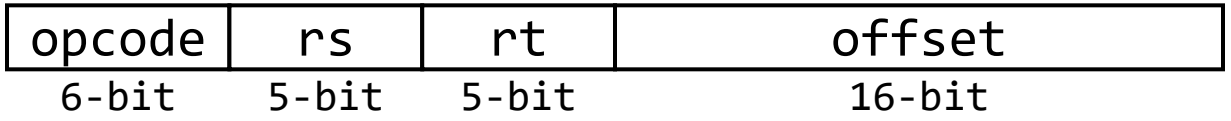


I-type

- opcode: there are immediate ALU instructions
  - addi, addiu, andi, ori, xori, slti, sltiu, lui, …
- Assembly and semantics
  - Corresponding assembly: **opcode rt rs immediate**
    - RF[rt] = RF[rs] op sign-extend(immediate)
      - sign-extend(1000…000) = concat(11…1,1000…000)
    - PC = PC + 4 (use the next instruction)
- What if you need to perform 32-bit immediate?
  - lui at 0xABCD // store the upper 16 bits
  - ori at 0x1234 // store the lower 16 bits

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

I-type Instructions – Memory ver.



I-type

- opcode: there are immediate memory instructions
  - lw, lh, lhu, lb, lbu, sw, sh, sb, …
  - // lw indicates load, sw indicates store

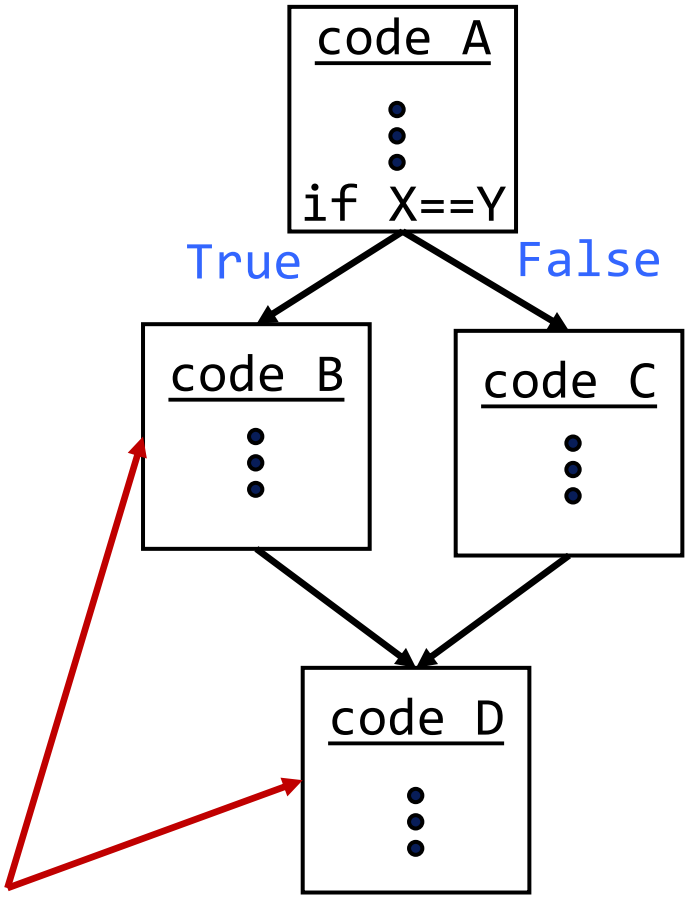
- Assembly and semantics
  - Corresponding assembler: **load/store rt offset(rs)**
    - RF[rt] = RF[RF[rs] + sign-extend(offset)] // load
    - MEM[RF[rs] + sign-extend(offset)]= RF[rt] // store
    - PC = PC + 4 (use the next instruction)

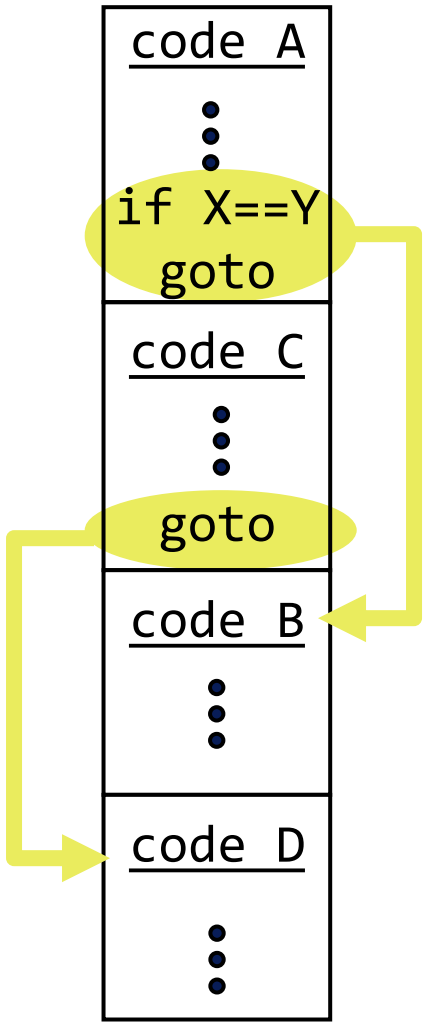
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Control Flow Instructions

Example code:

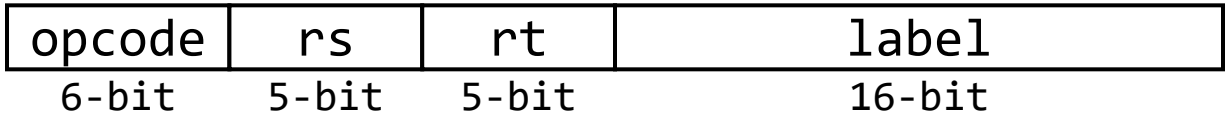
```
code A
if (X == Y){
        code B
}
else {
        code C
}
code D
```





© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

I-type Instructions – Control ver.



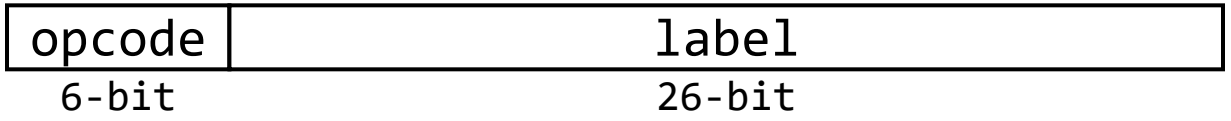
I-type

- opcode: there are eight immediate memory instructions
  - bne, beq, …
  - branch equal or not equal …

- Assembler and semantics
  - Corresponding assembly: **beq rs rt label**
    - target = (PC + 4) + sign-extend(label) x 4 // word-aligned
    - if (RF[rs] == RF[rt]) PC = target else PC = PC + 4
  - If you want to jump more than 18 bits (i.e., 16 bit + 2 bit) ➔ Utilize J-type (in the following slides)

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

J-type Instructions

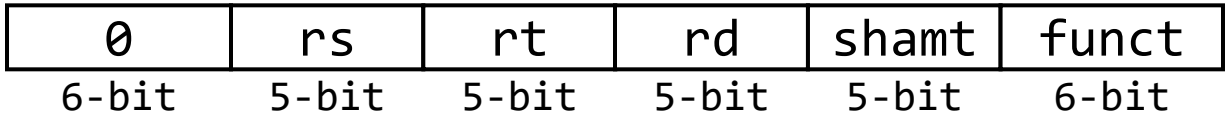


J-type

- opcode: there are eight immediate memory instructions
  - j, jal
- Assembler and semantics
  - target = (PC+4)[31:28]x2^{28} $|_{\text{bitwise-or}}$ zero-extend(label) x 4
  - // use the first four bits of PC and append label x 4
  - Corresponding assembly: **j label**
    - PC = target
  - Corresponding assembly: **jal label**
    - RF[ra] = PC + 4
      - save the next PC to ra (a dedicated register)
    - PC = target

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

R-type Instructions – Control ver.



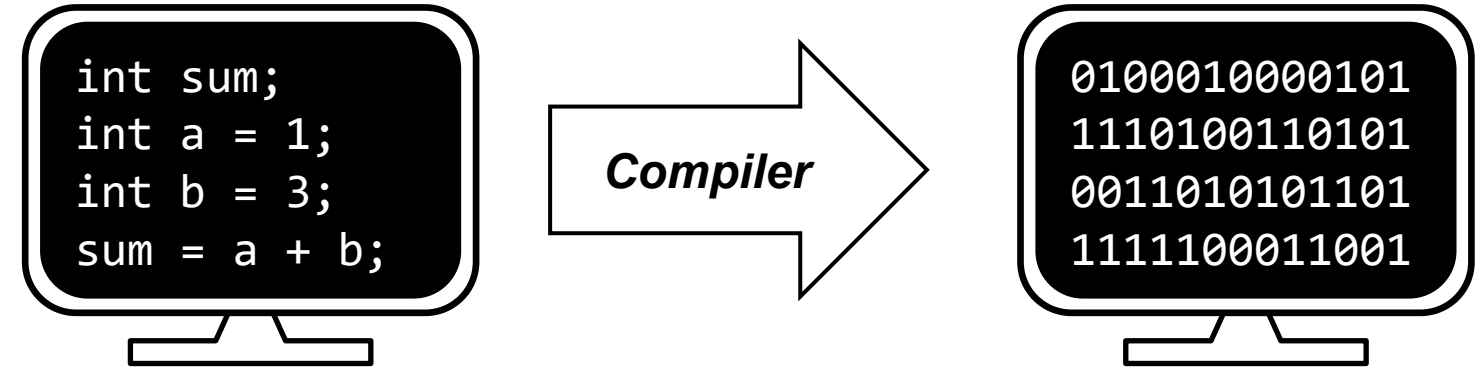
R-type

- There exists an R-type control instruction
  - `jr`
- Generally used along with `jal label` (upon a function call)
- Assembler and semantics
  - `Corresponding assembly: jr rs`
    - `PC = RF[rs]`

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Filling the Gap

- **The overall compilation toolchain translates your program into a computer-executable form**
  - Your program is human-understandable language
  - But, computers can execute binary instructions
  - The compiler is there to fill the gap!



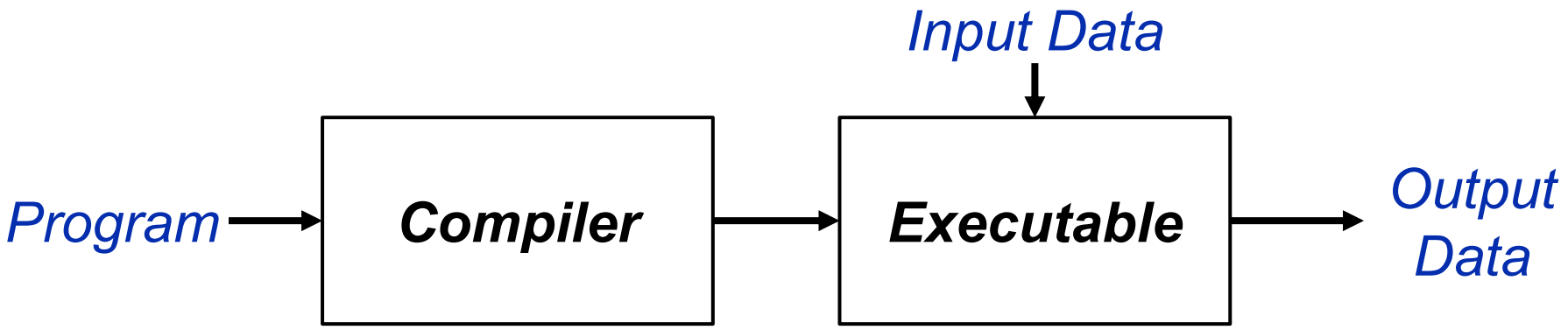
How you describe the program (Program)

What computer understands (Executable)

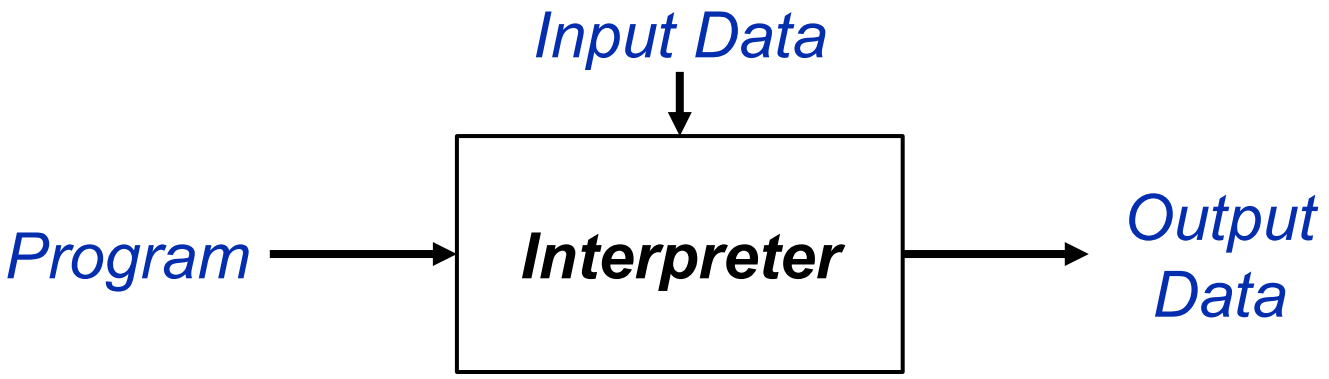
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Compiler vs. Interpreter

- **Compilers (The main focus)**



- **Interpreter**



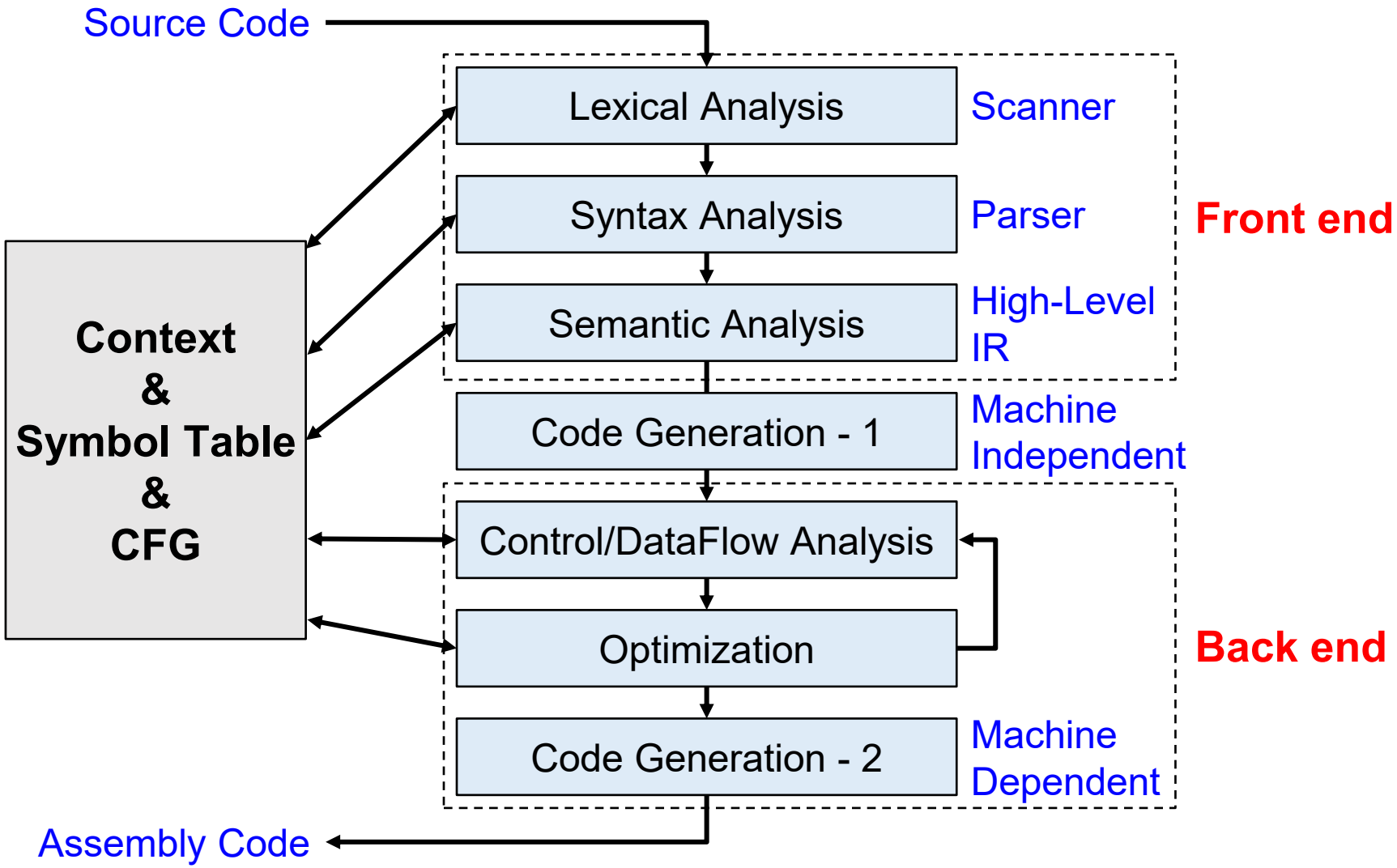
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Compiler

- **Front-End: Lets you program with high-level languages**
  - In 1970s, you had to use punch cards for programming
  - Now, we are in the surge of diverse programming languages
    - Fortran, C, C++, Java, Python, R, Tex, Html, …
- **Back-End: Let's you program without considering hardware-specific or optimization issues**
  - The internal computer architectures are hard for programmers to fully understand
  - The compiler optimizes the redundant codes

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

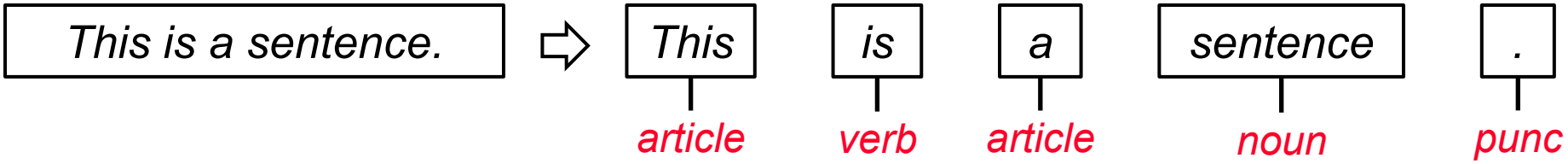
General Structure of a Modern Compiler



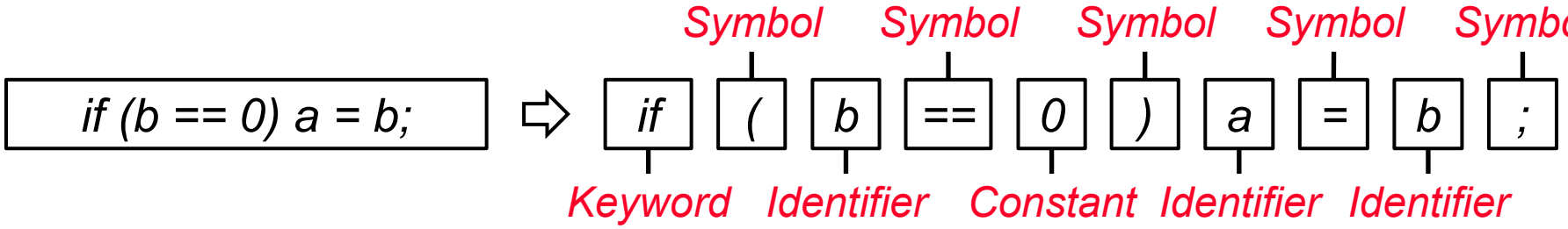
© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Lexical Analysis (Scanner)

- **Language: Recognizing words from sentences**



- **Program: Dividing programs into "tokens"**



© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

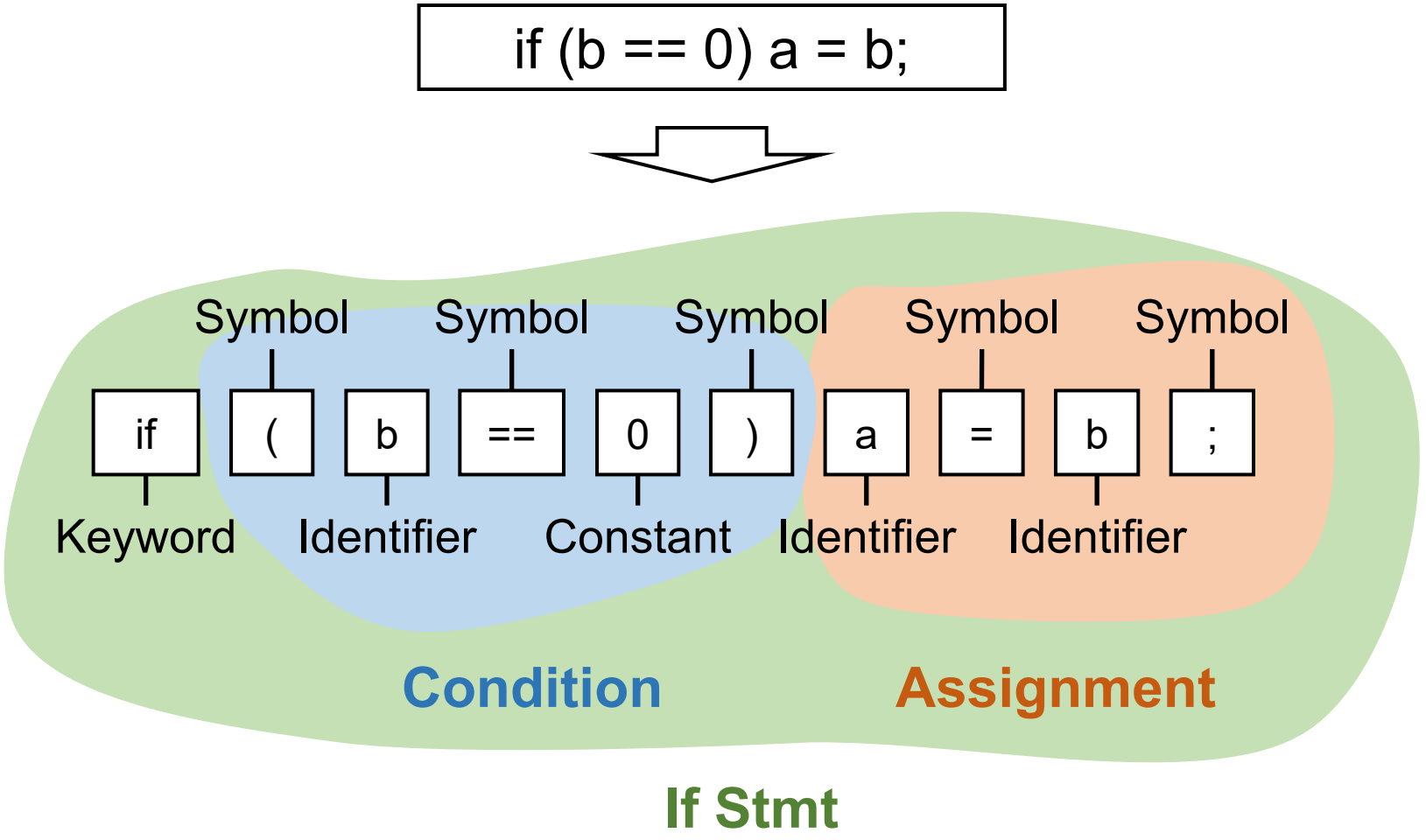
Lexical Analysis (Scanner)

- Extracts and identifies lowest-level lexical elements
  - Reserved words: for, if, switch
  - Identifiers: “i”, “j”, “table”
  - Constants: 3.14159, 17, “%d\n”
  - Punctuation symbols: “(“,”)”, “,”, “+”
- Removes non-grammatical elements – i.e. spaces, comments
- Implemented with a Finite State Automata (FSA)
  - Set of states – partial inputs
  - Transition functions to move between states

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Syntax Analysis (Parser)

- **Program: Determines the relationship between tokens**



© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Syntax Analysis (Parser)

- **Checks the program for syntactic correctness**
  - Framework for subsequent semantic processing

- **Lots of variations**
  - Hand-written recursive descent
  - Table-based (Top-down vs. Bottom-up)

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Semantic Analysis

- **Should support several distinct actions**
  - Check identifier definitions and validate the correctness
  - Type checking
  - Declaration scopes
  - Disambiguate overloaded operators
  - Translate from source to intermediate representation (IR)
  - Etc

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Optimization

- **Optimize code quality**
- **There are several optimization opportunities**
  - Identify constant values and compute them in advance
  - Remove unused variables
  - Compute loop-invariant variables
  - Remove recomputed variables
  - Etc

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Code Generation

- **Map machine independent intermediate representation to the target architecture**
- **Virtual to physical binding**
  - Instruction selection: best machine opcodes to implement generic opcodes
  - Register allocation: infinite virtual registers to N physical registers
  - Assembly emission
- **Machine assembly is our output, assembler, linker take over to create binary**

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim

Question?

© Lee 2026 -- Portions © Austin, Brehob, Falsafi, Hill, Hoe, Lipasti, Martin, Roth, Shen, Smith, Sohi, Tyson, Vijaykumar, Wenisch, Mutlu, Kim